Foldable & Traverse

Foldable/Traverse · Fold scores; traverse validated IDs · Build with Waqar

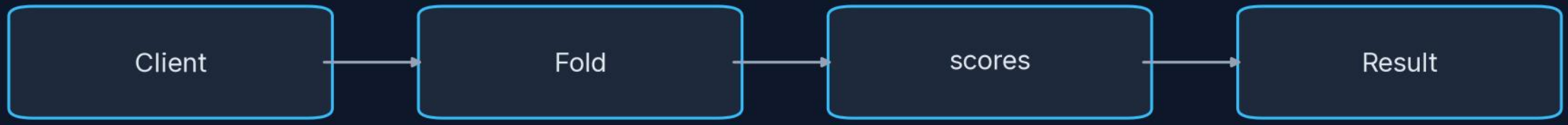
Today's Hook

- Problem: Fold scores; traverse validated IDs
- Pattern: Foldable/Traverse — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Fold scores; traverse validated IDs
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Foldable/Traverse: Fold scores; traverse validated IDs

Real-World Scenario

- Running example: Fold scores; traverse validated IDs
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.foldable.problem;

import java.util.*;

public final class NaiveFold {
  public static int sum(List<Integer> xs){ return xs.stream().mapToInt(i->i).sum(); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.foldable.problem;

import java.util.*;
import java.util.function.*;

final class Foldables {
  static int foldLeft(List<Integer> xs, int z, java.util.function.IntBinaryOperator op){
    int acc=z; for(int x: xs) acc=op.applyAsInt(acc,x); return acc;
  }
  static Optional<List<Integer>> traverse(List<String> ids, Function<String, Optional<Integer>> parse){
    List<Integer> out=new ArrayList<>();
    for(String id: ids){ var p=parse.apply(id); if(p.isEmpty()) return Optional.empty(); out.add(p.get()); }
    return Optional.of(out);
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.foldable.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Foldable/Traverse: scores + validated IDs ===");
    System.out.println("--- naive ---"); System.out.println(NaiveFold.sum(List.of(1,2,3)));
System.out.println("--- foldable ---");
System.out.println(Foldables.foldLeft(List.of(1,2,3), 0, Integer::sum));
System.out.println(Foldables.traverse(List.of("1","2"), s -> { try{return Optional.of(Integer.parseInt(s));}catch(Exception e){return Optional.empty();} }));
System.out.println(Foldables.traverse(List.of("1","x"), s -> { try{return Optional.of(Integer.parseInt(s));}catch(Exception e){return Optional.empty();} }));
  }
}
```

Walkthrough Output

```
=== Foldable/Traverse: scores + validated IDs ===
--- naive ---
6
--- foldable ---
6
Optional[[1, 2]]
Optional.empty
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Foldable/Traverse.

Related Patterns

- Contrast Foldable/Traverse with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Foldable/Traverse → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.